

Synopsis:

Currently we want to use Scalene-- which excels by:

- Line-by-line profiling of CPU, memory, and GPU usage.
- Differentiates between Python vs. native code execution.
- Estimates energy consumption based on CPU/GPU time.
- Identifies memory leaks and inefficient allocations.

Limited by:

- No direct power measurement: Estimates energy but does not read power sensors.
- Python-only focus: Cannot profile deep learning backends in C++ (like TensorFlow, PyTorch CUDA).
- *Lack of fine-grained energy data: Does not capture power fluctuations in real time.?*
- Cannot profile multiple processes: AI workloads involve multiple threads; Scalene focuses on the Python process.

What can we do?

CodeCarbon -- not specifically tracking hardware and runtime but making estimates based on historical data

is unique because it estimates the environmental impact of AI workloads rather than just profiling performance. It works by:

- Detecting CPU and GPU usage and estimating energy consumption.
- Using geographical data to determine the carbon intensity of electricity in different locations.
- Providing CO₂ emission estimates to help AI practitioners make energy-efficient choices.

Best Use Cases:

- Estimating the carbon footprint of *training* large ML models.
- Comparing energy consumption across cloud vs. local environments.
- Optimizing AI workloads to be more eco-friendly.

```
PS C:\Users\jaken\desktop\pythonsamples> python sample_script2.py
Starting computations...
Slow function result: 247475250000, Time: 0.0597s
Medium function computed, Time: 0.5479s
Fast function result: 333283335000, Time: 0.0010s
PS C:\Users\jaken\desktop\pythonsamples> python sample_script2.py
[codecarbon INFO @ 19:32:37] [setup] RAM Tracking...
[codecarbon INFO @ 19:32:37] [setup] CPU Tracking...
[codecarbon WARNING @ 19:32:37] No CPU tracking mode found. Falling back on CPU constant mode.
Windows OS detected: Please install Intel Power Gadget to measure CPU

[codecarbon INFO @ 19:32:39] CPU Model on constant consumption mode: Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz
[codecarbon INFO @ 19:32:39] [setup] GPU Tracking...
[codecarbon INFO @ 19:32:39] No GPU found.
[codecarbon INFO @ 19:32:39] >>> Tracker's metadata:
[codecarbon INFO @ 19:32:39] Platform system: Windows-10-10.0.19045-SP0
[codecarbon INFO @ 19:32:39] Python version: 3.12.1
[codecarbon INFO @ 19:32:39] CodeCarbon version: 2.8.3
[codecarbon INFO @ 19:32:39] Available RAM : 7.781 GB
[codecarbon INFO @ 19:32:39] CPU count: 8
[codecarbon INFO @ 19:32:39] CPU model: Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz
[codecarbon INFO @ 19:32:39] GPU count: None
[codecarbon INFO @ 19:32:39] GPU model: None
[codecarbon INFO @ 19:32:39] Saving emissions data to file C:\Users\jaken\desktop\pythonsamples\emissions.csv
Starting computations...
Slow function result: 247475250000, Time: 0.0544s
Medium function computed, Time: 0.0950s
Fast function result: 333283335000, Time: 0.0024s
[codecarbon INFO @ 19:32:39] Energy consumed for RAM : 0.000000 kWh. RAM Power : 2.9179658889770512 W
[codecarbon INFO @ 19:32:39] Energy consumed for all CPUs : 0.000001 kWh. Total CPU Power : 12.5 W
[codecarbon INFO @ 19:32:39] 0.000001 kWh of electricity used since the beginning.
C:\Users\jaken\AppData\Local\Programs\Python\Python312\Lib\site-packages\codecarbon\output_methods\file.py:52: FutureWarning: The behavior of DataFrame concatenation
with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the o
ehavior, exclude the relevant entries before the concat operation.
  df = pd.concat([df, pd.DataFrame.from_records([dict(total.values)])])
PS C:\Users\jaken\desktop\pythonsamples>
```

PyJoule:

Need *windows subsystem for linux* to run this as it needs low level hardware access:

- **Intel RAPL**
 - pyJoules uses the Intel "Running Average Power Limit" (RAPL) technology that estimates power consumption of the CPU, ram and integrated GPU.
- or **NVIDIA API**
 -

Decorate a function to measure its energy consumption

To measure the energy consumed by the machine during the execution of the function `foo()` run the following code:

```
from pyJoules.energy_meter import measure_energy

@measure_energy
def foo():
    # Instructions to be evaluated.

foo()
```

Output

The output structure of all domains are enabled including their respective units are as follows:

name	timestamp	tag	duration	package	dram	core	uncore	nvidia_gpu
type	datetime	str	ms	uJ	uJ	uJ	uJ	mJ

In my testing, evaluated CodeCarbon, PyJoules, and Scalene for energy profiling in AI workloads.

- CodeCarbon provides high-level energy and CO₂ emission estimates, making it great for tracking the environmental impact of models, but lacks fine-grained function-level insights.

- PyJoules, on the other hand, directly measures energy consumption using hardware power sensors like Intel RAPL and NVIDIA GPU APIs, allowing per-function energy tracking. However, PyJoules has limited Windows support, requiring Win Subsys Linux for accurate CPU power measurement.

- Scalene focuses on performance profiling, identifying slow or inefficient Python code while estimating energy usage indirectly through CPU/GPU time analysis.

Scalene and CodeCarbon can be used together for optimization and impact assessment.

Perun

- Measures energy consumption of Python scripts using Intel RAPL, ROCM-SMI, Nvidia-NVML, and psutil
- Monitor individual functions using decorators

Example of a perun report with benchmarking:

@monitor

```
PERUN REPORT
```

App name: torch_mnist
First run: 2023-08-22T17:44:34.927402
Last run: 2023-08-22T17:45:46.992693

RUN ID: 2023-08-22T17:45:46.992693

Round #	Host	RUNTIME	ENERGY	CPU_POWER	CPU_UTIL	GPU_POWER	GPU_MEM	DRAM_POWER
0	hkn0402.localdomain	52.988 s	24.379 kJ	202.854 W	0.865 %	234.184 W	4.281 GB	22.858 W
0	All	52.988 s	24.379 kJ	202.854 W	0.865 %	234.184 W	4.281 GB	22.858 W
1	hkn0402.localdomain	48.401 s	22.319 kJ	203.366 W	0.886 %	234.821 W	4.513 GB	22.798 W
1	All	48.401 s	22.319 kJ	203.366 W	0.886 %	234.821 W	4.513 GB	22.798 W
2	hkn0402.localdomain	48.258 s	22.248 kJ	203.339 W	0.884 %	234.720 W	4.513 GB	22.850 W
2	All	48.258 s	22.248 kJ	203.339 W	0.884 %	234.720 W	4.513 GB	22.850 W
3	hkn0402.localdomain	48.537 s	22.393 kJ	203.269 W	0.884 %	234.984 W	4.513 GB	22.968 W
3	All	48.537 s	22.393 kJ	203.269 W	0.884 %	234.984 W	4.513 GB	22.968 W
4	hkn0402.localdomain	48.416 s	22.323 kJ	203.408 W	0.888 %	234.626 W	4.513 GB	22.928 W
4	All	48.416 s	22.323 kJ	203.408 W	0.888 %	234.626 W	4.513 GB	22.928 W

Monitored Functions

Round #	Function	Avg Calls / Rank	Avg Runtime	Avg Power	Avg CPU Util	Avg GPU Mem Util
0	train	1	50.169±0.000 s	458.380±0.000 W	0.875±0.000 %	2.727±0.000 %
0	train_epoch	5	8.930±0.903 s	439.707±12.743 W	0.875±0.008 %	2.743±0.154 %
0	test	5	1.103±0.004 s	232.750±1.219 W	0.805±0.030 %	2.809±0.023 %
1	train	1	48.354±0.000 s	453.376±0.000 W	0.886±0.000 %	2.820±0.000 %
1	train_epoch	5	8.556±0.008 s	428.418±11.199 W	0.890±0.018 %	2.820±0.000 %
1	test	5	1.115±0.002 s	272.918±80.330 W	0.798±0.018 %	2.820±0.000 %
2	train	1	48.210±0.000 s	453.867±0.000 W	0.884±0.000 %	2.820±0.000 %
2	train_epoch	5	8.525±0.022 s	423.647±1.049 W	0.888±0.013 %	2.820±0.000 %
2	test	5	1.117±0.005 s	312.983±97.688 W	0.806±0.012 %	2.820±0.000 %
3	train	1	48.486±0.000 s	452.940±0.000 W	0.884±0.000 %	2.820±0.000 %
3	train_epoch	5	8.577±0.012 s	433.627±13.812 W	0.888±0.017 %	2.820±0.000 %
3	test	5	1.120±0.003 s	233.973±3.516 W	0.789±0.022 %	2.820±0.000 %
4	train	1	48.367±0.000 s	453.256±0.000 W	0.888±0.000 %	2.820±0.000 %
4	train_epoch	5	8.555±0.011 s	433.582±12.606 W	0.899±0.029 %	2.820±0.000 %
4	test	5	1.118±0.002 s	233.367±2.238 W	0.818±0.045 %	2.820±0.000 %

The application has been run 2 times. In total, it has used 0.062 kWh, released a total of 0.026 kgCO2e into the atmosphere, and

Not a great tool for windows

Recommendation -- Perun vs Scalene pros and cons:

Perun: Energy & Performance Versioning System

Best for: Tracking energy consumption over multiple executions and comparing different versions of a program.

Tracks:

- Energy usage (Joules, Watts)
- CPU & GPU power consumption
- Execution time & performance trends

Works with:

- Intel RAPL (CPU energy monitoring)
- Nvidia NVML (GPU energy tracking)
- AMD ROCM-SMI

Key Features:

- Tracks cumulative energy over multiple runs
- Works in distributed & MPI-based environments
- Stores historical energy usage for long-term optimization

Limitations:

- Requires hardware sensor access (not always available on Windows/VMs)
- Setup can be complex for function-level tracking

Scalene Performance Profiler

Best for: Code optimization by pinpointing bottlenecks in CPU, memory, and execution time.

Tracks:

- CPU usage (Python vs. C/C++ execution time)
- Memory allocation & leaks
- Estimated energy consumption (but not exact readings)

Works with:

- Any Python program
- No special hardware requirements

Key Features:

- Line-by-line profiling to identify energy-heavy functions
- Distinguishes between native vs. Python execution
- Estimates energy usage based on CPU time

Limitations:

- Energy estimates, not actual power readings
- No built-in long-term tracking for energy efficiency

My recommendation: Scalene to optimize speed & memory, and Perun to track actual energy savings. -- might mean that we don't have to fundamentally alter the scalene source code, rather use multiple tools in conjunction to gain a more well rounded view of what's going on.